

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ ФАКУЛЬТЕТ

ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1 по курсу «ООП» Тема: Управление заказами в
службе доставки.

Вариант 25 Выполнил: Чекушин В.С. К3241

Преподаватель: Николай Кочубеев

Санкт-Петербург 2025 г.

Оглавление

Структура проекта 3

Реализация базового класса и наследования 3

Паттерн Builder для создания предметов 3

Логика инвентаря и апгрейда..... 3

Unit-тестирование 4

Вывод 4

Структура проекта

Система управления заказами еды, построенная на модульной архитектуре с использованием пяти ключевых паттернов проектирования:

- **Singleton**: обеспечивает единую точку доступа к базе данных меню.
- **Strategy**: позволяет динамически менять алгоритм расчета стоимости (скидки/обычная цена).
- **Decorator**: используется для гибкого наложения дополнительных сборов (налоги, доставка, экспресс-сборы).
- **Factory Method**: инкапсулирует логику создания различных типов заказов.
- **Observer**: реализует систему уведомлений клиента об изменении статуса заказа.

Реализация Singleton (MenuDatabase)

Гарантирует, что список блюд загружается один раз и доступен глобально.

```
public class MenuDatabase
{
    private static MenuDatabase _instance;
    public List<EdaItem> items;

    private MenuDatabase() { /* инициализация списка */ }

    public static MenuDatabase GetInstance()
    {
        if (_instance == null) _instance = new MenuDatabase();
        return _instance;
    }
}
```

Паттерн Decorator (Расчет стоимости)

Позволяет добавлять стоимость услуг без изменения базового класса заказа.

```
public class TaxDecorator : OrderCostBase
{
    private OrderCostBase _prev;
    public TaxDecorator(OrderCostBase prev) => _prev = prev;
    public override double GetFinalCost() => _prev.GetFinalCost() * 1.2;
}

public class DeliveryFeeDecorator : OrderCostBase
{
    private OrderCostBase _prev;
    public DeliveryFeeDecorator(OrderCostBase prev) => _prev = prev;
    public override double GetFinalCost() => _prev.GetFinalCost() + 150;
}
```

Паттерн Observer (Уведомления)

Класс Order оповещает всех подписчиков (IOrderWatcher) при вызове метода ChangeStatus.

```
public void ChangeStatus(OrderStatus newStatus)
{
    Status = newStatus;
    foreach (var w in watchers)
        w.Update(newStatus.ToString());
}
```

Unit-тестирование

Проект покрыт тестами, проверяющими корректность работы паттернов, в частности точность расчетов при использовании декораторов и стратегий.

```
[Fact]
public void Test_StandardOrder_Calculation()
{
    var order = new StandardOrder();
    order.AddDish(new EdaItem { Name = "Test", Price = 100 });

    // (100 * 1.2 [налог]) + 150 [доставка] = 270
    double expected = 270;
    Assert.Equal(expected, order.CalculateTotal());
}
```

Вывод

- **Гибкость:** Добавление нового типа сбора (например, «Праздничная наценка») требует создания одного класса-декоратора без правки основной логики.
- **Полиморфизм в Strategy:** Заказ может менять поведение расчета цены «на лету» через свойство strategy.
- **Безопасность создания:** OrderFactory скрывает детали инициализации конкретных классов заказов от вызывающего кода.